



A03 - Búsqueda Exhaustiva y Aleatoria

Algoritmos Metaheurísticos

José Luis Chávez Gómez

3 de septiembre de 2026

1. Marco Teórico

Antes de recurrir a metaheurísticos poblacionales (algoritmos genéticos, enjambre de partículas, recocido simulado, etc.), es fundamental comprender el desempeño de los métodos de búsqueda directa más simples: aquellos que no usan ninguna estructura poblacional ni mecanismos evolutivos, sino que operan sobre un único punto o un conjunto de puntos independientes en el espacio de búsqueda $\Omega \subseteq \mathbb{R}^d$.

Estos métodos se clasifican, de forma general, en dos categorías:

- **Métodos de exploración global (ciegos):** no utilizan ninguna información sobre la estructura de f ; simplemente muestrean el dominio, ya sea de forma sistemática (búsqueda exhaustiva) o probabilística (búsqueda aleatoria). Su ventaja es que no pueden quedar atrapados en mínimos locales, pero su convergencia es lenta y su precisión está limitada por el número de muestras.
- **Métodos de explotación local:** parten de un punto y se mueven iterativamente hacia soluciones mejores en su vecindad inmediata, ya sea siguiendo el gradiente de f (gradiente descendente) o probando vecinos aleatorios y aceptando solo mejoras (Hill Climbing). Convergen rápido cuando el paisaje es favorable (unimodal, convexo), pero son vulnerables a quedar atrapados en óptimos locales cuando f es multimodal [1].

Como señala Simon [1], esta dicotomía entre exploración y explotación es precisamente la tensión central que los algoritmos evolutivos y metaheurísticos posteriores intentan balancear de forma más sofisticada, combinando mecanismos de ambos tipos. Los cuatro algoritmos implementados en esta actividad representan, en cierto sentido, los dos extremos de ese espectro en su forma más pura, antes de introducir población, memoria o mecanismos de escape de óptimos locales.

1.1. Búsqueda Exhaustiva

Discretiza el dominio Ω en una malla uniforme de puntos y evalúa f en cada uno, quedándose con el mínimo encontrado:

$$x^* \approx \arg \min_{x \in G} f(x), \quad G \subset \Omega \text{ malla finita} \quad (1)$$

Su precisión está acotada por la resolución de la malla, y su costo computacional crece exponencialmente con la dimensión (n^d evaluaciones), lo que lo hace impráctico más allá de 2 o 3 dimensiones [3].

1.2. Búsqueda Aleatoria

En lugar de una malla determinista, muestrea N puntos de una distribución uniforme sobre Ω :

$$x_k \sim \mathcal{U}(\Omega), \quad k = 1, \dots, N \quad (2)$$

y se queda con el mejor de los N puntos evaluados. A diferencia de la malla, no está limitado a posiciones fijas, lo que puede ofrecer mejor precisión con el mismo presupuesto de evaluaciones, aunque sin garantía alguna [2].

1.3. Gradiente Descendente con Diferencias Finitas

Cuando no se cuenta con la derivada analítica de f , el gradiente se aproxima numéricamente mediante diferencias finitas centrales:

$$\frac{\partial f}{\partial x_i}(x) \approx \frac{f(x + h e_i) - f(x - h e_i)}{2h} \quad (3)$$

y se actualiza iterativamente en la dirección de mayor descenso:

$$x_{t+1} = x_t - \alpha \nabla f(x_t) \quad (4)$$

donde α es la tasa de aprendizaje. Este método converge muy rápido en funciones convexas y bien condicionadas, pero es sensible a la elección de α y puede divergir o quedar atrapado en mínimos locales en funciones multimodales [4].

1.4. Hill Climbing

Parte de un punto x_t y genera un vecino aleatorio cercano:

$$x' = x_t + \epsilon, \quad \epsilon \sim \mathcal{N}(0, \sigma^2 I) \quad (5)$$

aceptando el movimiento únicamente si mejora la solución actual:

$$x_{t+1} = \begin{cases} x' & \text{si } f(x') < f(x_t) \\ x_t & \text{en otro caso} \end{cases} \quad (6)$$

A diferencia del gradiente descendente, no requiere información direccional alguna, pero por la misma razón queda igualmente expuesto a mínimos locales, siendo el ejemplo canónico de búsqueda local pura en la literatura de inteligencia artificial [5].

2. Algoritmos Implementados

Los cuatro algoritmos se probaron sobre cuatro funciones objetivo de la actividad A01, seleccionadas para cubrir distintos tipos de paisaje: **Sphere** (unimodal, tazón), **Rastrigin** y **Ackley** (multimodales), y **Zakharov** (fuerte interacción entre variables), todas en $d = 2$ para permitir su visualización y animación.

2.1. Búsqueda Exhaustiva

Código:

```
1
2 import numpy as np
3 import matplotlib.pyplot as plt
4 from basics import Objective_function, Animation
5
6 def exhaustive_search(obj, n_points, batch_size=None):
7     """
8     Búsqueda exhaustiva sobre una malla uniforme del dominio de obj.
9
10    Parameters
11    -----
12    obj : Objective_function
13        Instancia con obj.d, obj.lb, obj.ub y obj.eval(X) definidos.
14    n_points : int
15        Número de puntos por eje de la malla.
16    batch_size : int, opcional
17        Cuántos puntos de la malla se agrupan por cuadro de animación.
18        Si es None, se calcula para tener aproximadamente 50 cuadros.
19
20    Returns
21    -----
22    best_x : np.array de forma (d,)
23    best_f : float
24    lotes : list de (X_lote, ig_local)
25        Listo para pasarle directo a Animation.
26    """
27    ejes = [np.linspace(obj.lb[i], obj.ub[i], n_points) for i in range(obj.d)]
28
29    mesh = np.meshgrid(*ejes)
30    X = np.vstack([m.ravel() for m in mesh])
31
32    valores = obj.eval(X)
33
34    best_idx = np.argmin(valores)
35    best_x = X[:, best_idx]
36    best_f = valores[best_idx]
37
38    if batch_size is None:
39        batch_size = max(1, X.shape[1] // 50)
40
41    lotes = []
42    for start in range(0, X.shape[1], batch_size):
43        end = start + batch_size
44        X_lote = X[:, start:end]
45        f_lote = valores[start:end]
46        ig_local = np.argmin(f_lote)
47        lotes.append((X_lote, ig_local))
48
49    return best_x, best_f, lotes
50
51
```

```

52 if __name__ == "__main__":
53     from rastringin import Rastringin
54
55     obj = Rastringin(dim=2)
56     n_points = 40
57
58     best_x, best_f, lotes = exhaustive_search(obj, n_points=n_points)
59
60     print("Mejor x:", best_x)
61     print("Mejor f(x):", best_f)
62     print("N mero de lotes (cuadros de animaci n):", len(lotes))
63
64     anim = Animation(animation_type='3d', name="Exhaustive Search - Rastringin")
65     X0, ig0 = lotes[0]
66     anim.initialize_animation(obj, X0, ig0)
67     for X_lote, ig in lotes[1:]:
68         anim.update_animation(obj, X_lote, ig)
69
70     plt.show()

```

Resultados (mejor $f(x)$ encontrado, malla de 40×40 puntos):

Función	$f(x^*)$
Sphere	$3,446995 \times 10^{-2}$
Rastrigin	4,225349
Ackley	4,101178
Zakharov	0,000000

Cuadro 1: Resultados de búsqueda exhaustiva sobre las 4 funciones.

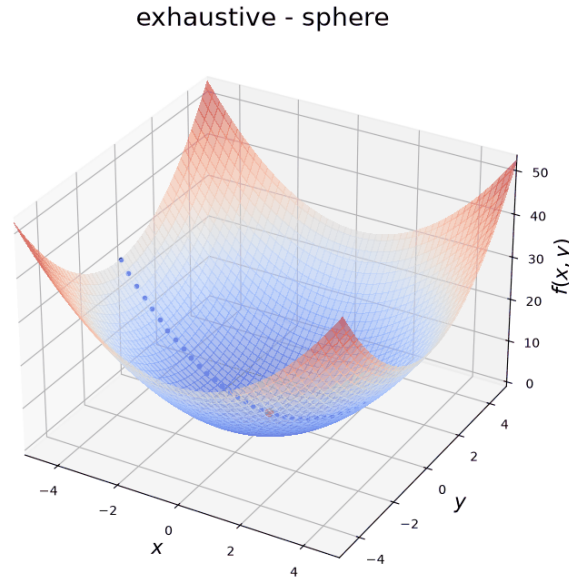


Figura 1: Cuadro representativo de la animación de búsqueda exhaustiva sobre Sphere.

2.2. Búsqueda Aleatoria

Código:

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from basics import Objective_function, Animation
4
5 def random_search(obj, n_samples, batch_size=None, seed=None):
6
7     if seed is not None:
8         np.random.seed(seed)
9
10    lb = np.array(obj.lb).reshape(-1, 1)
11    ub = np.array(obj.ub).reshape(-1, 1)
12    X = lb + np.random.rand(obj.d, n_samples) * (ub - lb)
13
14    valores = obj.eval(X)
15
16    best_idx = np.argmin(valores)
17    best_x = X[:, best_idx]
18    best_f = valores[best_idx]
19
20    if batch_size is None:
21        batch_size = max(1, n_samples // 50)
22
23    lotes = []
24    for start in range(0, n_samples, batch_size):
25        end = start + batch_size
26        X_lote = X[:, start:end]
27        f_lote = valores[start:end]
28        ig_local = np.argmin(f_lote)
29        lotes.append((X_lote, ig_local))
30
31    return best_x, best_f, lotes
32
33
34 if __name__ == "__main__":
35     from sphere import Sphere
36
37     obj = Sphere(dim=2)
38     n_samples = 1600
39
40     best_x, best_f, lotes = random_search(obj, n_samples=n_samples, seed=42)
41
42     print("Mejor x:", best_x)
43     print("Mejor f(x):", best_f)
44     print("Número de lotes (cuadros de animación):", len(lotes))
45
46     anim = Animation(animation_type='3d', name="Random Search - Sphere")
47     X0, ig0 = lotes[0]
48     anim.initialize_animation(obj, X0, ig0)
49     for X_lote, ig in lotes[1:]:
50         anim.update_animation(obj, X_lote, ig)
51
52     plt.show()
```

Resultados (mejor $f(x)$ encontrado, 1600 muestras, seed=42):

Función	$f(x^*)$
Sphere	$1,165799 \times 10^{-2}$
Rastrigin	1,599993
Ackley	4,162210
Zakharov	$6,547096 \times 10^{-2}$

Cuadro 2: Resultados de búsqueda aleatoria sobre las 4 funciones.

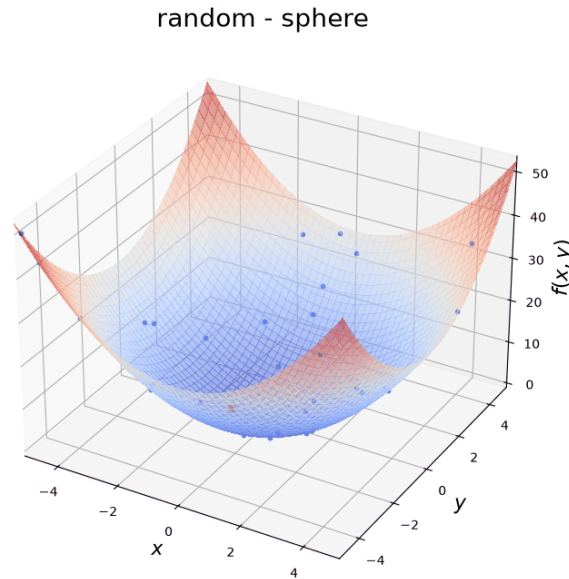


Figura 2: Cuadro representativo de la animación de búsqueda aleatoria sobre Sphere.

2.3. Gradiente Descendente con Diferencias Finitas

Código:

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from basics import Objective_function, Animation
4
5 def gradient_descent_fd(obj, alpha, iteraciones, h=1e-5, seed=None, x_inicial=
  None):
6     """
7     Gradiente descendente donde el gradiente se aproxima numéricamente
8     mediante diferencias finitas centrales (no se deriva la función
9     analíticamente).
10
11     Parameters
12     -----
13     obj : Objective_function
14         Instancia con obj.d, obj.lb, obj.ub y obj.function(x) definidos.
15     alpha : float
16         Tasa de aprendizaje.
17     iteraciones : int
18         Número de pasos de descenso.
19     h : float, opcional

```

```

19     Tama o del paso para la diferencia finita central.
20     seed : int, opcional
21         Semilla para el punto inicial aleatorio.
22     x_inicial : np.array, opcional
23         Si se da, se usa como punto de partida en vez de uno aleatorio.
24
25     Returns
26     -----
27     best_x : np.array de forma (d,)
28     best_f : float
29     lotes : list de (X_lote, ig_local)
30         Un lote por iteraci n (X_lote de forma (d, 1)), listo para Animation.
31     """
32     if seed is not None:
33         np.random.seed(seed)
34
35
36     if x_inicial is None:
37         lb = np.array(obj.lb)
38         ub = np.array(obj.ub)
39         x = lb + np.random.rand(obj.d) * (ub - lb)
40     else:
41         x = np.array(x_inicial, dtype=float)
42
43     lotes = []
44     best_x = x.copy()
45     best_f = obj.function(x)
46
47     for it in range(iteraciones):
48         grad = np.zeros(obj.d)
49         for i in range(obj.d):
50             x_mas = x.copy()
51             x_menos = x.copy()
52             x_mas[i] += h
53             x_menos[i] -= h
54             grad[i] = (obj.function(x_mas) - obj.function(x_menos)) / (2 * h)
55
56         x = x - alpha * grad
57         x = np.clip(x, obj.lb, obj.ub)
58
59         f_x = obj.function(x)
60         X_lote = x.reshape(obj.d, 1) # una sola columna: un punto por
61             iteraci n
62         lotes.append((X_lote, 0))
63
64         if f_x < best_f:
65             best_f = f_x
66             best_x = x.copy()
67
68     return best_x, best_f, lotes
69
70 if __name__ == "__main__":
71     from sphere import Sphere
72
73     obj = Sphere(dim=2)
74
75     best_x, best_f, lotes = gradient_descent_fd(
76         obj, alpha=0.1, iteraciones=100, h=1e-5, seed=42
77     )
78
79     print("Mejor x:", best_x)
80     print("Mejor f(x):", best_f)

```

```

81 print("N mero de iteraciones (cuadros de animaci n):", len(lotes))
82
83 anim = Animation(animation_type='3d', name="Gradient Descent (FD) - Sphere")
84 X0, ig0 = lotes[0]
85 anim.initialize_animation(obj, X0, ig0)
86 for X_lote, ig in lotes[1:]:
87     anim.update_animation(obj, X_lote, ig)
88
89 plt.show()

```

Resultados (200 iteraciones, α ajustado por función, seed=42):

Función	α	$f(x^*)$
Sphere	0.1	$3,954417 \times 10^{-38}$
Rastrigin	0.01	$2,978394 \times 10^1$
Ackley	0.01	$1,975220 \times 10^1$
Zakharov	0.001	6,111972

Cuadro 3: Resultados de gradiente descendente con diferencias finitas sobre las 4 funciones.

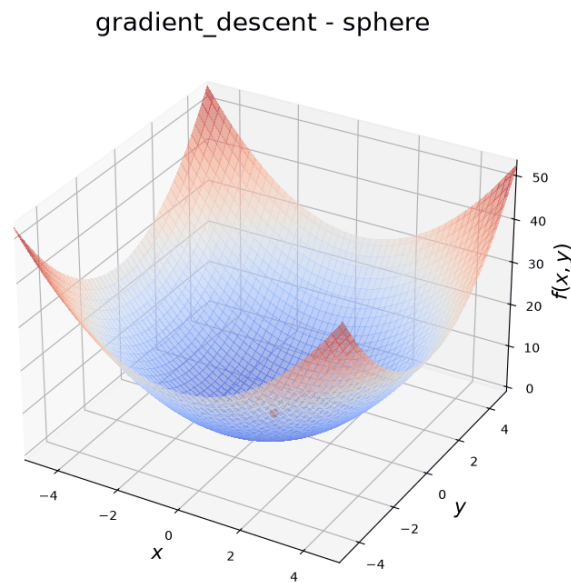


Figura 3: Cuadro representativo de la animación de gradiente descendente sobre Sphere.

2.4. Hill Climbing

Código:

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from basics import Objective_function, Animation
4
5 def hill_climbing(obj, iteraciones, step_size=0.1, seed=None, x_inicial=None):
6     """
7     Hill Climbing simple: en cada iteraci n genera UN vecino aleatorio

```

```

8     cercano al punto actual, y se mueve ah  solo si mejora.
9
10    Parameters
11    -----
12    obj : Objective_function
13          Instancia con obj.d, obj.lb, obj.ub y obj.function(x) definidos.
14    iteraciones : int
15          N mero de iteraciones (intentos de vecino).
16    step_size : float, opcional
17          Desviaci n est ndar del ruido gaussiano usado para generar el vecino.
18    seed : int, opcional
19          Semilla para reproducibilidad.
20    x_inicial : np.array, opcional
21          Si se da, se usa como punto de partida en vez de uno aleatorio.
22
23    Returns
24    -----
25    best_x : np.array de forma (d,)
26    best_f : float
27    lotes : list de (X_lote, ig_local)
28          Un lote por iteraci n (X_lote de forma (d, 1)), listo para Animation.
29          Nota: aqu  X_lote siempre es el punto ACTUAL (se haya movido o no
30          en esa iteraci n), para que la animaci n muestre tambi n los
31          intentos donde se qued  quieto.
32    """
33    if seed is not None:
34        np.random.seed(seed)
35
36    lb = np.array(obj.lb)
37    ub = np.array(obj.ub)
38
39
40    if x_inicial is None:
41        x = lb + np.random.rand(obj.d) * (ub - lb)
42    else:
43        x = np.array(x_inicial, dtype=float)
44
45    f_x = obj.function(x)
46
47    lotes = []
48    best_x = x.copy()
49    best_f = f_x
50
51    for it in range(iteraciones):
52        vecino = x + np.random.normal(0, step_size, obj.d)
53        vecino = np.clip(vecino, lb, ub)  # no salirse del dominio
54
55        f_vecino = obj.function(vecino)
56
57        if f_vecino < f_x:
58            x = vecino
59            f_x = f_vecino
60
61        X_lote = x.reshape(obj.d, 1)
62        lotes.append((X_lote, 0))
63
64        if f_x < best_f:
65            best_f = f_x
66            best_x = x.copy()
67
68    return best_x, best_f, lotes
69
70

```

```

71 if __name__ == "__main__":
72     from sphere import Sphere
73     obj = Sphere(dim=2)
74
75     best_x, best_f, lotes = hill_climbing(
76         obj, iteraciones=200, step_size=0.2, seed=42
77     )
78
79     print("Mejor x:", best_x)
80     print("Mejor f(x):", best_f)
81     print("Numero de iteraciones (cuadros de animacion):", len(lotes))
82
83     anim = Animation(animation_type='3d', name="Hill Climbing - Sphere")
84     X0, ig0 = lotes[0]
85     anim.initialize_animation(obj, X0, ig0)
86     for X_lote, ig in lotes[1:]:
87         anim.update_animation(obj, X_lote, ig)
88
89     plt.show()

```

Resultados (200 iteraciones, step_size=0.2, seed=42):

Función	$f(x^*)$
Sphere	$8,833723 \times 10^{-4}$
Rastrigin	$2,595497 \times 10^1$
Ackley	$1,977631 \times 10^1$
Zakharov	$1,153187 \times 10^{-3}$

Cuadro 4: Resultados de Hill Climbing sobre las 4 funciones.

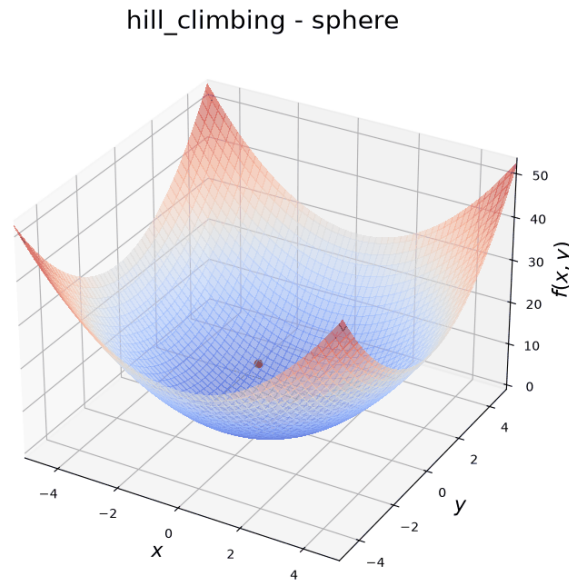


Figura 4: Cuadro representativo de la animación de Hill Climbing sobre Sphere.

2.5. Script de Experimentos

Para automatizar las 16 combinaciones (4 algoritmos $\times$ 4 funciones) y generar las animaciones correspondientes en un solo proceso, se desarrolló el siguiente script:

```
1 import os
2 import numpy as np
3 import matplotlib.pyplot as plt
4 from matplotlib.animation import FuncAnimation, PillowWriter
5 from PIL import Image
6
7 from sphere import Sphere
8 from rastringin import Rastringin
9 from ackley import Ackley
10 from zakharov import Zakharov
11
12 from exhaustive_search import exhaustive_search
13 from random_search import random_search
14 from gradient_descent_fd import gradient_descent_fd
15 from hill_climbing import hill_climbing
16
17 OUTPUT_DIR = "resultados"
18 os.makedirs(OUTPUT_DIR, exist_ok=True)
19
20 funciones = {
21     "sphere": Sphere,
22     "rastringin": Rastringin,
23     "ackley": Ackley,
24     "zakharov": Zakharov,
25 }
26
27 alphas_gd = {
28     "sphere": 0.1,
29     "rastringin": 0.01,
30     "ackley": 0.01,
31     "zakharov": 0.001,
32 }
33
34 def crear_animacion_gif(obj, lotes, ruta_archivo, titulo):
35     fig = plt.figure(figsize=(8, 8))
36     obj.draw3d(resolution=100, name=titulo)
37     ax = fig.gca()
38
39     X0, ig0 = lotes[0]
40     puntos, = ax.plot(X0[0, :], X0[1, :], obj.eval(X0), '.b')
41     mejor, = ax.plot([X0[0, ig0]], [X0[1, ig0]], [obj.function(X0[:, ig0])],
42                     '.r', markersize=12)
43
44     def actualizar(frame):
45         X_lote, ig = lotes[frame]
46         puntos.set_data(X_lote[0, :], X_lote[1, :])
47         puntos.set_3d_properties(obj.eval(X_lote))
48         mejor.set_data([X_lote[0, ig]], [X_lote[1, ig]])
49         mejor.set_3d_properties([obj.function(X_lote[:, ig])])
50         return puntos, mejor
51
52     anim = FuncAnimation(fig, actualizar, frames=len(lotes), interval=150)
53     anim.save(ruta_archivo, writer=PillowWriter(fps=6))
54     plt.close(fig)
55
56 def correr_algoritmo(nombre_alg, obj, nombre_func):
57     if nombre_alg == "exhaustive":
58         return exhaustive_search(obj, n_points=40)
59     elif nombre_alg == "random":
```

```

60     return random_search(obj, n_samples=1600, seed=42)
61 elif nombre_alg == "gradient_descent":
62     alpha = alphas_gd[nombre_func]
63     return gradient_descent_fd(obj, alpha=alpha, iteraciones=200, h=1e-5,
64                               seed=42)
65 elif nombre_alg == "hill_climbing":
66     return hill_climbing(obj, iteraciones=200, step_size=0.2, seed=42)
67 algoritmos = ["exhaustive", "random", "gradient_descent", "hill_climbing"]
68 resumen = []
69
70 for nombre_func, ClaseFuncion in funciones.items():
71     obj = ClaseFuncion(dim=2)
72
73     for nombre_alg in algoritmos:
74         print(f"Corriendo {nombre_alg} sobre {nombre_func}...")
75
76         best_x, best_f, lotes = correr_algoritmo(nombre_alg, obj, nombre_func)
77
78         titulo = f"{nombre_alg} - {nombre_func}"
79         ruta = os.path.join(OUTPUT_DIR, f"{nombre_alg}_{nombre_func}.gif")
80         crear_animacion_gif(obj, lotes, ruta, titulo)
81
82         resumen.append({
83             "algoritmo": nombre_alg,
84             "funcion": nombre_func,
85             "best_x": best_x,
86             "best_f": best_f,
87         })
88
89 print("\n=== RESUMEN ===")
90 for r in resumen:
91     print(f"{r['algoritmo']:18s} | {r['funcion']:12s} | best_f = {r['best_f']:.6e}")

```

3. Resultados Comparativos

Algoritmo	Función	$f(x^*)$
Búsqueda Exhaustiva	Sphere	$3,446995 \times 10^{-2}$
Búsqueda Aleatoria	Sphere	$1,165799 \times 10^{-2}$
Gradiente Descendente	Sphere	$3,954417 \times 10^{-38}$
Hill Climbing	Sphere	$8,833723 \times 10^{-4}$
Búsqueda Exhaustiva	Rastrigin	4,225349
Búsqueda Aleatoria	Rastrigin	1,599993
Gradiente Descendente	Rastrigin	$2,978394 \times 10^1$
Hill Climbing	Rastrigin	$2,595497 \times 10^1$
Búsqueda Exhaustiva	Ackley	4,101178
Búsqueda Aleatoria	Ackley	4,162210
Gradiente Descendente	Ackley	$1,975220 \times 10^1$
Hill Climbing	Ackley	$1,977631 \times 10^1$
Búsqueda Exhaustiva	Zakharov	0,000000
Búsqueda Aleatoria	Zakharov	$6,547096 \times 10^{-2}$
Gradiente Descendente	Zakharov	6,111972
Hill Climbing	Zakharov	$1,153187 \times 10^{-3}$

Cuadro 5: Resultados de las 16 combinaciones algoritmo–función.

4. Conclusión

Los resultados evidencian que ningún algoritmo domina de forma universal: el desempeño relativo de cada método depende directamente del tipo de paisaje de la función objetivo, confirmando la teoría revisada en el marco teórico.

En **Sphere**, función convexa y unimodal, el gradiente descendente con diferencias finitas obtuvo un resultado prácticamente exacto ($f(x^*) \approx 4 \times 10^{-38}$), muy por encima de los demás métodos, ya que contar con información direccional confiable permite converger de forma directa al óptimo sin necesidad de explorar el resto del dominio.

Sin embargo, este mismo método fue el de **peor desempeño relativo** en las funciones multimodales (Rastrigin y Ackley), donde tanto el gradiente descendente como Hill Climbing quedaron muy por detrás de los métodos ciegos (búsqueda exhaustiva y aleatoria). Esto confirma lo señalado en el marco teórico: los métodos que solo explotan localmente (siguiendo el gradiente o aceptando únicamente movimientos que mejoran) quedan atrapados en el primer mínimo local cercano a su punto de arranque aleatorio, mientras que los métodos que exploran el dominio completo, aunque sea a ciegas, tienen mayor probabilidad de encontrar una región cercana al óptimo global.

En **Zakharov**, función con fuerte interacción entre variables pero unimodal, la búsqueda exhaustiva obtuvo el mejor resultado numérico ($f(x^*) = 0$), aunque esto se debe en parte a que uno de los puntos de su malla cayó muy cerca del óptimo teórico $x^* = (0, 0)$; Hill Climbing tuvo un desempeño notablemente bueno ($1,15 \times 10^{-3}$), mientras que el gradiente descendente rindió pobremente (6,11), probablemente porque la tasa de aprendizaje utilizada resultó insuficiente para recorrer la distancia entre un punto de arranque aleatorio distante y el óptimo dentro del número de iteraciones asignado — un recordatorio de que este método requiere un ajuste cuidadoso de α específico para cada función, a diferencia de los métodos ciegos que no requieren ningún parámetro de este tipo.

En conjunto, este ejercicio ilustra el compromiso fundamental entre exploración y explotación

que motiva el desarrollo de metaheurísticos más sofisticados: un método puramente exploratorio nunca refina lo suficiente su solución, mientras que uno puramente explotador queda condenado a la calidad de su punto de partida. Los algoritmos evolutivos y poblacionales que se abordarán en actividades posteriores del curso buscan precisamente combinar ambos mecanismos —múltiples puntos de búsqueda, operadores de exploración global y explotación local— para superar esta limitación.

Referencias

- [1] Simon, D. (2013). *Evolutionary Optimization Algorithms: Biologically-Inspired and Population-Based Approaches to Computer Intelligence*. John Wiley & Sons.
- [2] Yang, X.-S. (2010). *Nature-Inspired Metaheuristic Algorithms* (2nd ed.). Luniver Press.
- [3] Talbi, E.-G. (2009). *Metaheuristics: From Design to Implementation*. John Wiley & Sons.
- [4] Engelbrecht, A. P. (2007). *Computational Intelligence: An Introduction* (2nd ed.). John Wiley & Sons.
- [5] Russell, S., & Norvig, P. (2020). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson.
- [6] Surjanovic, S., & Bingham, D. (2013). *Virtual Library of Simulation Experiments: Test Functions and Datasets*. Simon Fraser University. Recuperado de <https://www.sfu.ca/~ssurjano/>